Autoévaluation

Exercice - Définissez l'architecture front-end

Un dernier doute avant l'envoi de vos livrables ?

Pour vérifier la qualité de votre travail :

- cochez les cases ci-dessous : elles indiquent que vous avez bien pris en compte chaque indicateur de réussite ;
- renseignez, si besoin, la colonne "Notes" avec des commentaires sur vos livrables / vos étapes. Ils seront des points de discussion avec votre mentor pendant votre session de bilan / soutenance.

Quand toutes les cases de ce document seront cochées, vous pourrez déposer vos livrables sur la plateforme.

Bonne réussite !

Étape	Indicateurs de réussite de l'activité	Notes

Étape 1 – Analysez le code existant et repérez les problèmes	▣ J'ai analysé tout le starter code
	  ▣ J'ai lancé l'app avec `npm run dev`
	  ▣ J'ai observé le fonctionnement de l'app (Je suis capable de faire une démo de l'app en session.)
	▣ J'ai identifié les problèmes dans le starter code
	  ▣ J'ai catégorisé les problèmes (structure, duplication, typage, placement)
	  ▣ J'ai priorisé les problèmes selon leur impact.
	▣ J'ai rédigé mon analyse claire du code dans un fichier `notes-architecture.md`.
	  ▣ J'ai inclus des preuves référencées dans le fichier, par ex. un fichier ou une ligne, une capture d'écran.	

Étape 2 – Concevez une nouvelle architecture adaptée	▣ J'ai proposé une arborescence réaliste
	  ▣ J'ai choisi un design pattern adapté
	  ▣ J'ai identifié les blocs (ex. pages, components, etc.)
	  ▣ J'ai évité de la sur-structuration	

	☐ J'ai schématisé l'arborescence avec un outil comme draw.io, Whimsical ou Figma ☒ J'ai décrit l'arborescence dans `notes-architecture.md`. ☒ J'ai explicité les rôles et les responsabilités de chaque dossier	

Étape 3 – Refactorisez les composants existants en appliquant l'architecture	☒ J'ai séparé les composants
	  ☒ J'ai identifié les bouts de code qu'on peut sortir dans un composant réutilisable pour éviter la duplication de code
	  ☒ J'ai utilisé le(s) composant(s) réutilisable(s)
	  ☒ J'ai évité des composants mal découpés
	☒ J'ai refactoré la base React en suivant mon architecture proposée
	  ☒ Les imports/exports et les noms des composants sont mis à jour
	☒ J'ai suivi les bonnes pratiques de développement en React tout au long du refactoring
	  ☒ J'ai testé que le build est OK avec `npm run dev`
	  ☒ J'ai fait des commits atomiques et messages clairs
	☒ Il n'y a aucun "any" dans mon code	

Étape 4 – Externalisez la gestion des données dans un Custom Hook	▣ J'ai créé un Custom Hook.
	  ▣ Le hook centralise toutes les données
	  ▣ Le hook est importé dans les composants
	▣ J'ai vérifié le bon fonctionnement du hook
	▣ Je continue de suivre les bonnes pratiques de développement en React (testing, commits)	

Étape 5 – Documentez l'architecture et validez-la	▣ J'ai produit une structure claire et documentée dans le fichier `ARCHITECTURE.md`
	▣ Dans l'architecture que je propose :
	  ▣ `composants/` UI uniquement
	  ▣ `pages/` composants smart effectuant les récupérations de données
	  ▣ `hooks/` centralise l'accès aux données
	  ▣ `models/` avec interfaces TypeScript (zéro `any`)
	▣ Mon arborescence est cohérente et lisible pour un autre dev (ou bien mon mentor)	

Autoévaluation

Exercice - Développez le front-end en utilisant React

Un dernier doute avant l'envoi de vos livrables ?

Pour vérifier la qualité de votre travail :

- cochez les cases ci-dessous : elles indiquent que vous avez bien pris en compte chaque indicateur de réussite ;
- renseignez, si besoin, la colonne "Notes" avec des commentaires sur vos livrables / vos étapes. Ils seront des points de discussion avec votre mentor pendant votre session de bilan / soutenance.

Quand toutes les cases de ce document seront cochées, vous pourrez déposer vos livrables sur la plateforme.

Bonne réussite !

Étape	Indicateurs de réussite de l'activité	Notes

Étape 1 – Améliorez le dashboard existant	▣ Le dashboard conforme aux spécifications et aux maquettes (graphique + navigation)
	  ▣ Les données sont agrégées correctement
	  ▣ Le bar chart s'affiche bien
	▣ J'ai vérifié la responsivité du dashboard sur :
	  ▣ Desktop
	  ▣ Mobile	

Étape 2 – Améliorez la page détail et ajoutez la navigation	▣ La page détail conforme aux spécifications et aux maquettes (graphique + navigation)
	  ▣ Les bons KPI (médailles, athlètes, ex.) s'affichent
	  ▣ La bonne évolution des médailles s'affiche
	▣ J'ai vérifié la navigation fluide entre les pages
	  ▣ Si l'utilisateur fait clic → route country
	▣ J'ai utilisé les hooks `useParams` pour récupérer l'ID et `useData` pour trouver le pays correspondant.	

Étape 3 – Gérez les erreurs de navigation et d'affichage	▣ J'ai vérifié que le pays existe. Dans le cas contraire, j'affiche la page d'erreur not-found
	▣ J'ai implémenté la gestion d'un état de chargement
	▣ J'ai mis en place la gestion d'erreurs avec message/redirect :
	  ▣ ID invalide
	  ▣ mauvaise URL
	  ▣ données manquantes	

Étape 4 – Finalisez l'application et documentez-la sur GitHub	▣ J'ai vérifié que l'application fonctionne bien aux spécifications
	  ▣ J'ai vérifié la responsivité de chaque page (desktop et mobile)
	▣ J'ai nettoyé mon code
	  ▣ pas de `console.log`
	  ▣ duplication limitée
	  ▣ pas de `any`
	  ▣ Tout le code est propre et lisible (p.ex: les variables sont correctement nommées pour améliorer la lisibilité, il n'y a pas de code mort, etc.)
	▣ J'ai rédigé un README complet (installation, scripts, structure, limites)	

	▣ Je peux expliquer mes choix et mes actions à un autre dev (ou bien mon mentor)
	▣ J’ai suivi les bonnes pratiques de développement en React au fur et à mesure du projet
	  ▣ J’ai fait des commits Git clairs et fréquents
	  ▣ J’ai respecté globalement les guides de Vite et de React	